Name : Pooja Kumari
ROLL : 22F1000470
EMAIL : 22f1000470@ds.study.iitm.ac.in
I am a diploma level student of IITM BS.

Title: Influencer Engagement and Sponsorship Coordination Platform

1. Introduction: The Influencer Engagement and Sponsorship Coordination Platform connects sponsors and influencers for seamless advertising campaigns. Sponsors can create campaigns and manage ad requests, while influencers can engage, negotiate terms, and manage their profiles, streamlining influencer marketing and sponsorship processes.

2. Data Models:

- User Model: Stores user details (name, email, password, and role such as Admin, Sponsor, or Influencer).
- Campaign Model: Represents campaigns created by sponsors, including name, description, start/end dates, budget, visibility, and goals.
- AdRequest Model: Represents ad requests made by sponsors, including requirements, payment amount, status, and associated campaign and influencer.
- Req_from_inf : Stores negotiations between sponsors and influencers, including requested payment amounts.
- Relationships:
    - User → Campaign**:** One-to-Many (`sponsor_id`)
    - User → AdRequest**:** One-to-Many (`influencer_id`)
    - Campaign → AdRequest**:** One-to-Many (`campaign_id`).

3. System Architecture: The system follows a client-server architecture. The client-side is built using Vue.js, providing an interactive user interface for admin and users. On the server-side, Flask is used to handle API requests and interactions with the database.

4. Functionality:

- Campaign Management: Sponsors can create, edit, and delete campaigns. Campaigns can have multiple ad requests.
- Ad Request Management: Sponsors can send, view, and update ad requests to influencers. Influencers can accept or reject ad requests.
- Negotiation System: Sponsors and influencers can negotiate payment amounts through a dedicated system.
- Role-Based Access: Admins can manage users and monitor all campaigns and ad requests.
- Notifications: Notifications are sent to users through Celery and Redis for important updates.
- Email Notifications: Monthly reports are sent to users using Mailhog and Celery tasks.

5. User Interfaces:

Admin Dashboard: Manages users, campaigns, and platform activity.
Sponsor Dashboard: Creates campaigns, sends ad requests, and tracks ad requests.
Influencer Dashboard: Manages ad requests, updates profiles, and negotiates terms.

6. Important API resource endpoints:

- POST /login: Authenticates a user and starts a session.
- POST /register: Registers a new user with role-based access.
- POST /campaigns: Creates a new campaign.
- GET /ad-requests: Retrieves all ad requests for a specific user.
- POST /generate-csv: Triggers a Celery job to generate a CSV file of campaigns.

6. Scheduling Daily Reminder Jobs: Other key features of the app is the implementation of daily reminder jobs. Each evening, a scheduled job is executed by the system to initiate the daily reminder process. This job is to re-engage users who have not recently visited the platform or viewed ad_request, for them the system generates personalized alerts. These alerts are delivered through communication channels like webhook notifications.

7. Monthly Entertainment Report: Another valuable feature is the generation and delivery of a comprehensive Monthly Entertainment Report. This report provides users with insights into their interactions with the platform throughout the previous month. At the beginning of each month, a scheduled job is initiated within the system to generate the Monthly Entertainment Report. The HTML-based email report which contains a summary of their campaigns and presents it in a clear and engaging format.

8. User-Triggered Async Job: The app empowers users with the ability to initiate user-triggered asynchronous jobs for exporting campaign-related data in CSV format. The exported CSV file includes the following information: Campaign Name,Description,Start_Date, end_Date, Budget and Visibility. Users can trigger the asynchronous export job by clicking a designated button. The export job is managed using Celery. Upon successful completion of the export, users receive an alert informing them that their requested CSV file is downloaded.

9. Implementing Caching for Enhanced API Performance: caching is used to optimize the API performance and reduce the load on the server and to maintain data accuracy and freshness, cache expiry mechanisms are used. Used Redis, to implement caching effectively.

Video link:
https://drive.google.com/file/d/17Vdo-KMlQoCKxxXgO5NceH4SGDGpL0NL/view?usp=sharing